E. Generative Modeling

E.1 Generative Modeling: Basic concepts

E.2 Generative Modeling: Variational models

E.3 Generative Modeling: Diffusion models I

E.4 Generative Modeling: Diffusion models II

E.5 Generative Modeling: Structured (conditional) generation

- Why we need conditional generation
- Conditional VAEs
- Conditional Diffusion models

Tiam Heydari, PhD
Postdoctoral Fellow, UBC

From "make a sample" to "make this kind of sample": Structured (conditional) generation

- So far, our generative models have been unconditional. They can generate realistic samples from the overall data distribution:

$$x_{new} \sim p_{\text{data}}(x)$$

- Structure (c) is information that we attach to each data example and supply to the generator:

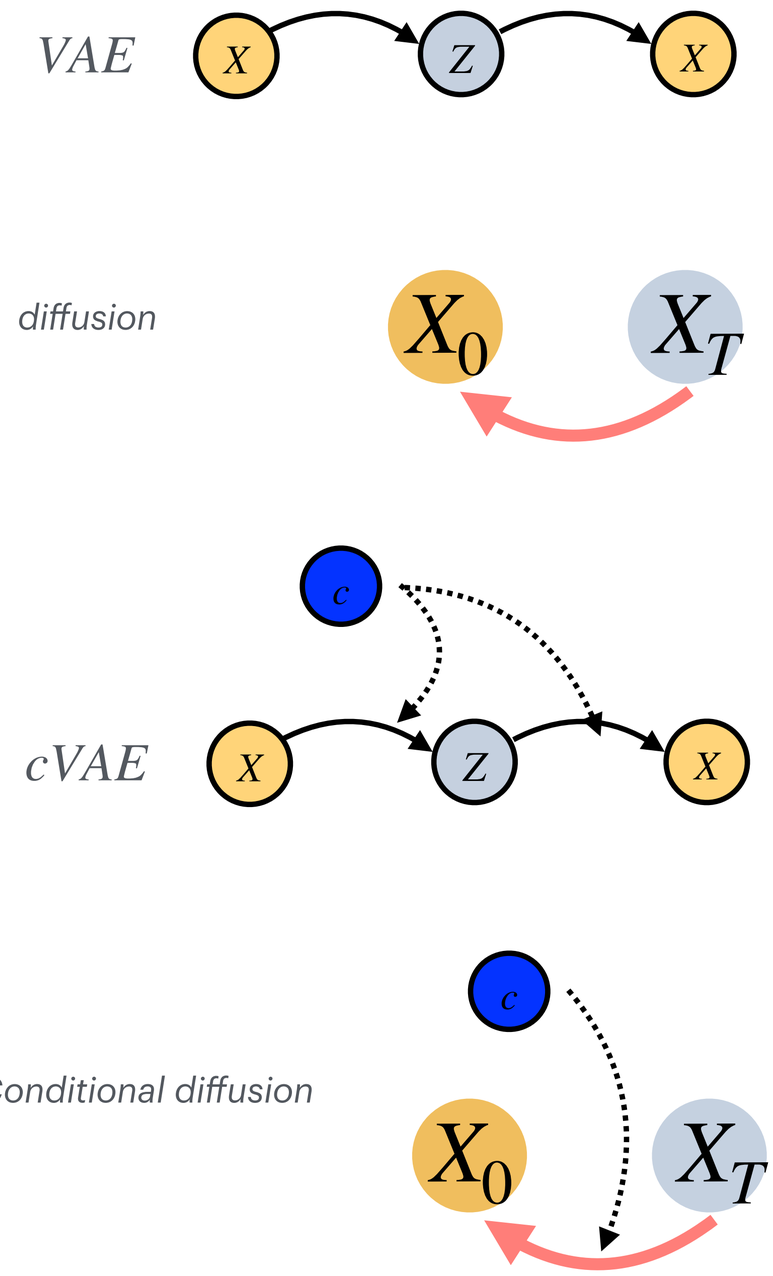
  $c =$ label, treatment, cell type, donor, time, prompt, or desired property.

- but they do not know what kind of sample we want. For example, they may generate a realistic cell, image, or molecule, but they cannot reliably generate:

  "a T cell treated with IL-2" or "a molecule with a desired property."

- The good news is that we can easily add this information c to the VAE and diffusion frameworks we already developed. Our new goal is:

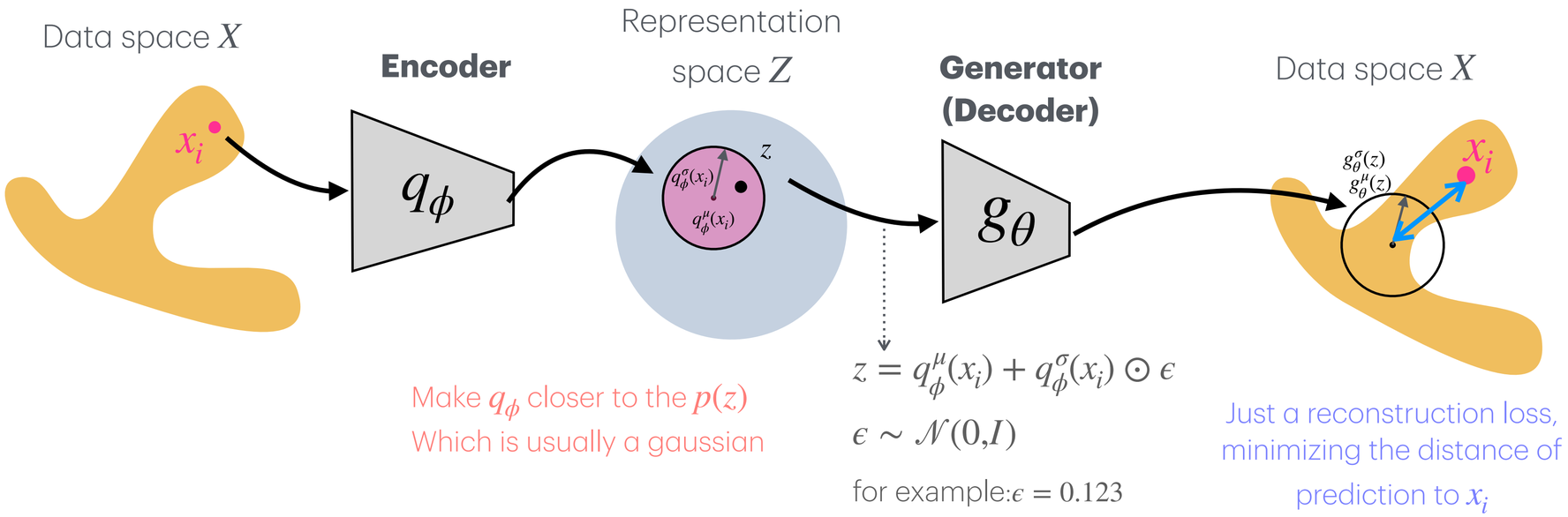
$$x_{new} \sim p_{\text{data}}(x \mid c)$$



Reminder: VAE

Variational Auto Encoder (**VAE**) in a glance:

Model:



Training:

$$\text{Maximizing } \mathscr{L}_{\text{ELBO}}(x_i) = \overbrace{\mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x_i \mid z)\right]}^{\uparrow} - \overbrace{D_{\text{KL}}\left(q_\phi(z \mid x_i) \| p(z)\right)}^{\downarrow}$$

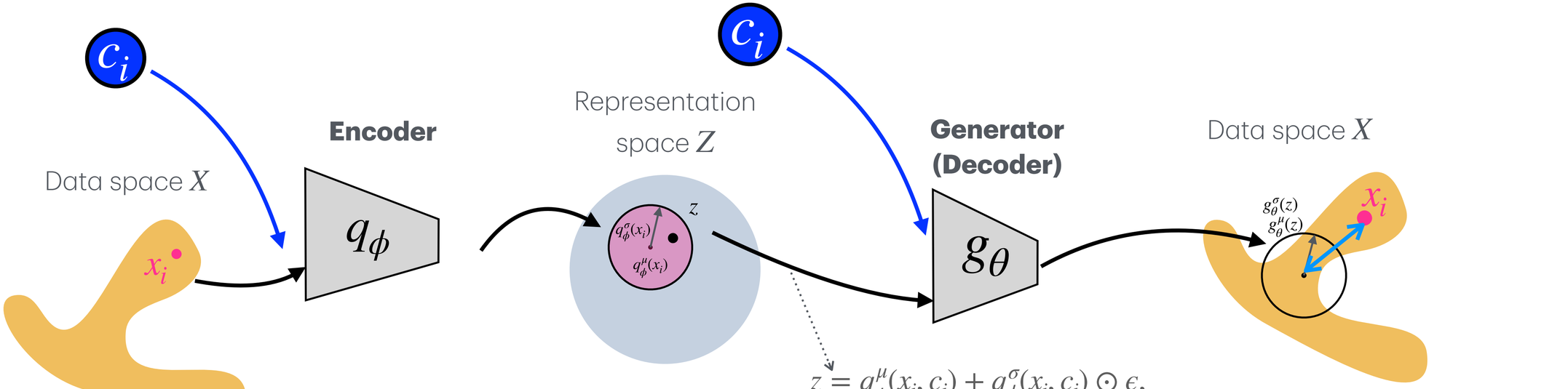
Conditional VAE (cVAE): same as VAE, but now tell it what kind of sample to make

- Now our training is based on $\{x_i, c_i\}$ where $c_i \in \mathbb{R}^{d_c}$ is vector describing some information about x_i. That's it.
- **For training** we will feed c_i to both q_ϕ and g_θ. **For generation** we sample from random z and a desired c.

Training:

$$\text{Maximizing } \mathscr{L}_{\text{ELBO}}(x_i, c_i) = \mathbb{E}_{z \sim q_\phi(z|x_i,c_i)}\left[\log p_\theta(x_i \mid z, c_i)\right] - D_{\text{KL}}\left(q_\phi(z \mid x_i, c_i) \,\|\, p(z)\right)$$

Model:



Make q_ϕ closer to the $p(z)$
Which is usually a gaussian

$$z = q_\phi^\mu(x_i, c_i) + q_\phi^\sigma(x_i, c_i) \odot \epsilon,$$

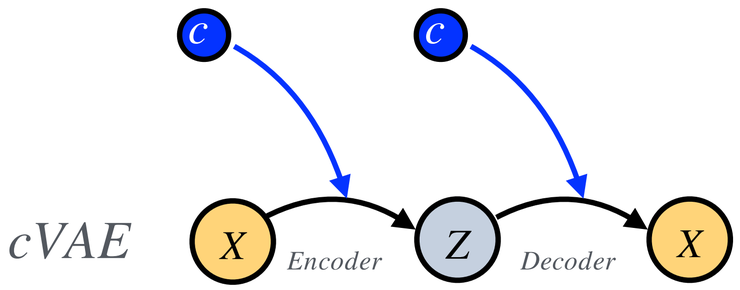
$$\epsilon \sim \mathcal{N}(0, I)$$

for example: $\epsilon = 0.123$

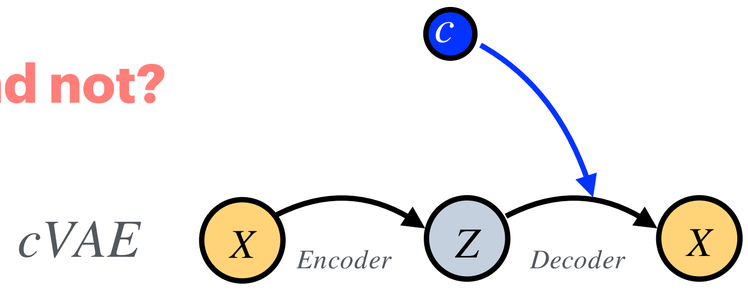
Just a reconstruction loss, minimizing the distance of prediction to x_i

Paus Point ||

Question: The decoder already receives c_i. Why should the encoder see it as well?



And not?



Answer:

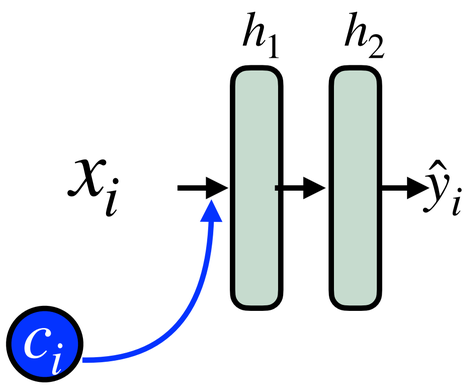
- Knowing c_i helps the *encoder* know/identify which part of x_i is already explained by the condition.
- The *encoder* can theoretically subtract that information from x_i.
- The latent space Z can then focus on the <u>remaining variation not explained by c_i</u>

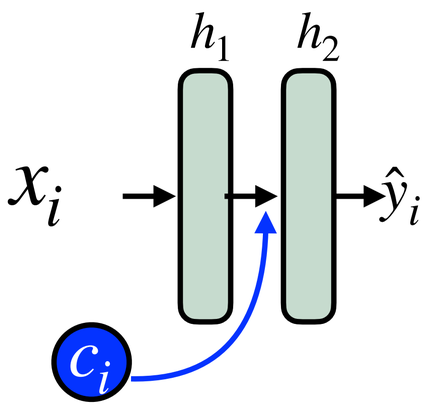
*** note that:** The KL term encourages z not to duplicate information already provided by c_i, but does not guarantee perfect independence!

Paus Point ||

The insight is simple: Where c_i enters changes the forward computation but c_i can affect learning in every layer.

Consider a simple 2 layer MLP (layers h_1 and h_2):





Forward:

$h_1 = f_1(x_i, c_i)$

h_1, h_2, $\hat{y}_i$ can all use c_i directly.

$h_1 = f_1(x_i)$

h_1 does not directly see c_i.

Backward:

c_i also changes the gradients used to update every layer.

Because the final prediction depends on c_i, the updates to h_1 can still change with c_i.

Early injection: c_i affects early layers explicitly in the forward pass too.

Late injection: c_i affects early layers implicitly through backpropagation.

Recipe Card: **Conditional VAEs (cVAEs)**

Type of Learning: **Conditional generative learning**

Data set: **Paired data**(x_i, c_i)

Data

1- Load libraries: Basics + PyTorch

2-Load your Dataset: $D = \{(x_i, c_i)\}_{i=1}^{N}$

-$x_i =$ data example, $c_i =$ condition associated with that example .

3-Now set up the DataLoader for training/validation/testing:

-Each batch must return both the sample and its condition: $(x_{\text{batch}}, c_{\text{batch}})$.

NN model

4-Building the cVAE neural network:

-The VAE will have 4 main parts:

A) [Encoder Network] or q_ϕ:

Input, $[x_i, c_i]$→[NN layers of choice based on data]→⋯→[hidden representation] →

(Example NNs: MLP for vectors, CNN for images, GNN for graphs, DeepSet for sets)

B) [Latent Parameter Layer]:

$$\rightarrow \left(\mu_i = \mu_\phi(x_i, c_i), \qquad \sigma_i = \sigma_\phi(x_i, c_i)\right) \rightarrow$$

C) [Reparameterization Layer]:

$$z_i = \mu_i + \sigma_i \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0,I) .$$

ϵ is selected from a distribution $\mathcal{N}$, it has <u>vector</u> from and not learnable.

D) [MLP Decoder Network]:

$[z_i, c_i]$→[NN layers of choice based on data]→⋯→ out, $\hat{x}_i$

-The whole VAE network will be:

$(x_i, c_i) \rightarrow$ **[Encoder, NN]** $\rightarrow (\mu_i, \sigma_i) \rightarrow z_i$

$(z_i, c_i) \rightarrow$ **[Decoder, NN]** $\rightarrow \hat{x}_i$.

-Note: σ and μ will have the same dimensions as Z. Because each element of Z will be sampled based on the mean and variance that are encoded for that element in σ and μ.

Training

5-Define a proper loss function.

-same as for a VAE, but encoder and decoder receive c_i:

$$\mathscr{L}_{\text{cVAE}} = \text{Reconstruction}(\hat{x}_i, x_i) + \beta D_{\text{KL}}\left(q_\phi(z \mid x_i, c_i) \| p(z)\right)$$

-where $\hat{x}_i = g_\theta(z_i, c_i)$.

-Reconstruction loss depends on the data structure

-Usually the prior is normal, but can also vary:

$$p(z) = \mathcal{N}(0,I)$$

-Add regularization (to network) to prevent overfitting

-The loss will be the sum of all relevant losses

6-Define the train and test functions:

-Each batch now contains paired data: $(x_{\text{batch}}, c_{\text{batch}})$.

-β Should be tuned (usually between 0-1)

-then:

$(x_i, c_i) \rightarrow$ *Encoder* $\rightarrow (\mu_i, \sigma_i) \rightarrow z_i$ **and** $(z_i, c_i) \rightarrow$ *Decoder* $\rightarrow \hat{x}_i$.

7-Define training and run the training:

-Training and validation are the same as for a standard VAE.

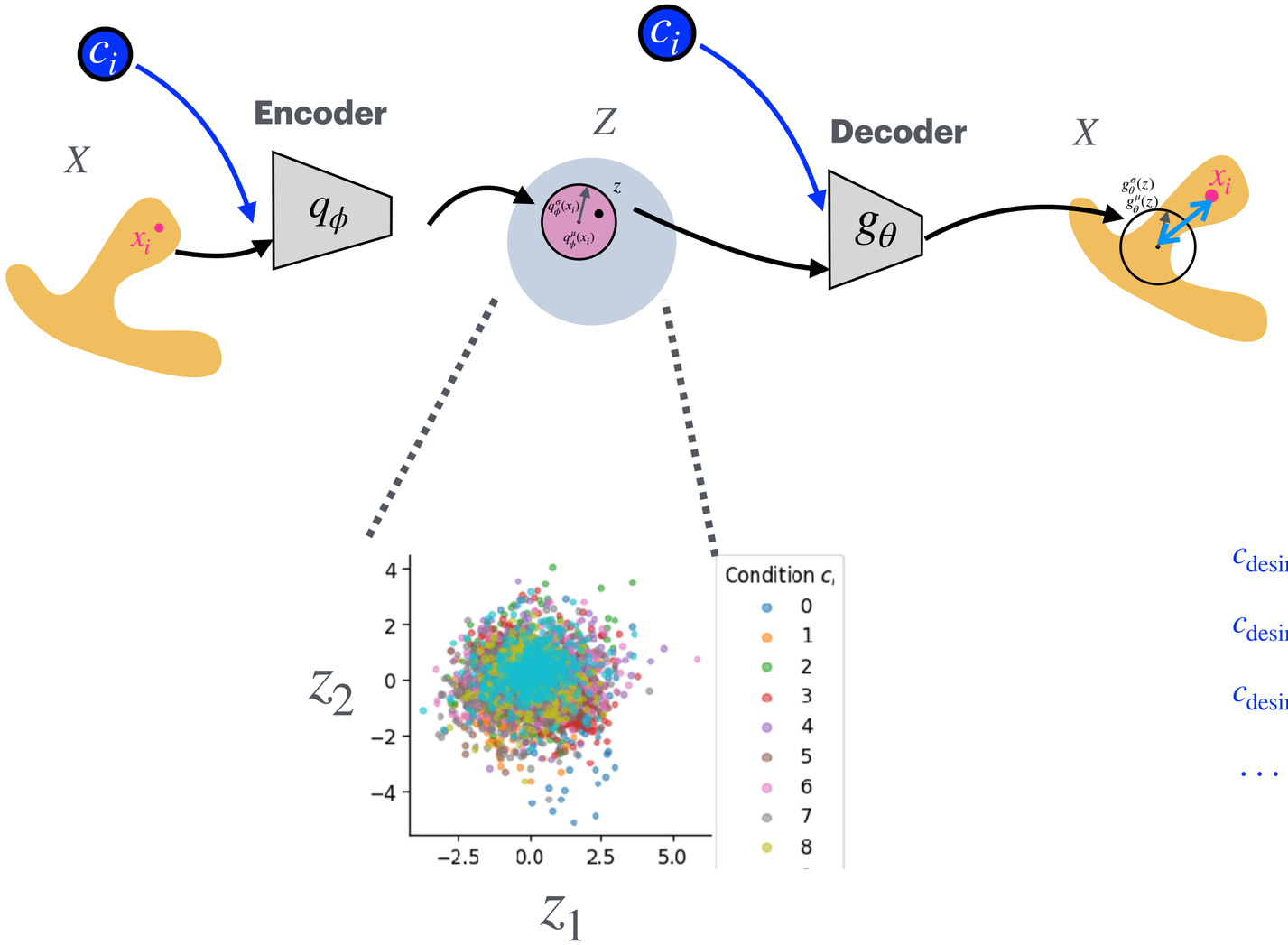
8- Generate a sample:

-Choose a desired condition: c_{desired}

-Sample random latent noise: $z \sim \mathcal{N}(0,I)$

-Decode: $x_{\text{new}} = g_\theta(z, c_{\text{desired}})$.

Conditional VAE (cVAE): Example on MNIST dataset



New samples via conditioning:

We traverse the 2D latent space by scanning a grid of points:

$$z_{new} = (z_1, z_2)$$

Decoding each point via z_new and c_{desired} :

$$\hat{x}_{\text{new}} = g_\theta(z, c_{\text{desired}}) \, .$$

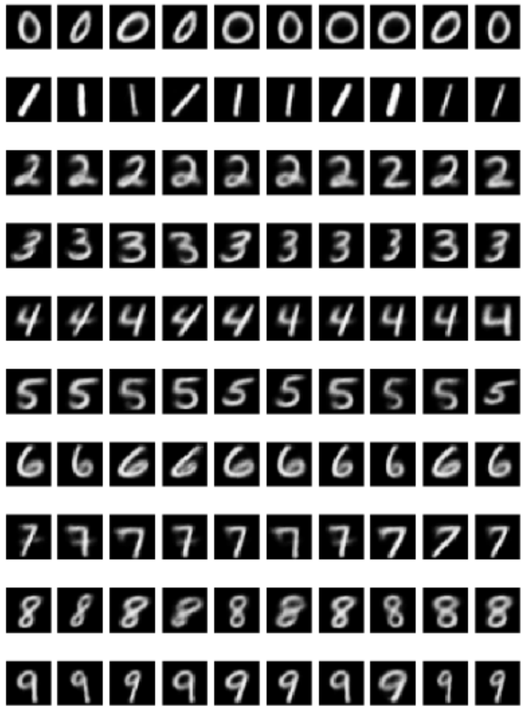
Traverse $z \rightarrow$

$c_{\text{desired}} = [1,0,0,0,0,0,0,0,0,0]$

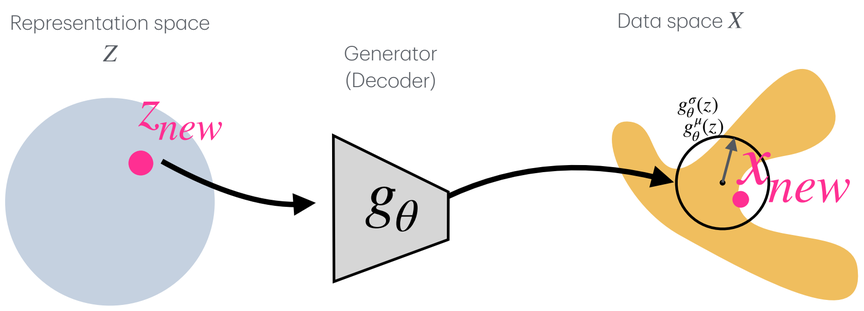
$c_{\text{desired}} = [0,1,0,0,0,0,0,0,0,0]$

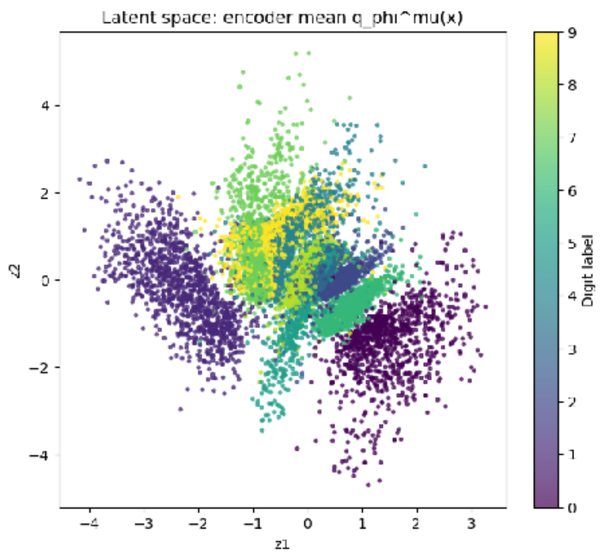
$c_{\text{desired}} = [0,0,1,0,0,0,0,0,0,0]$

$\ldots$



Compare previous example with VAE:





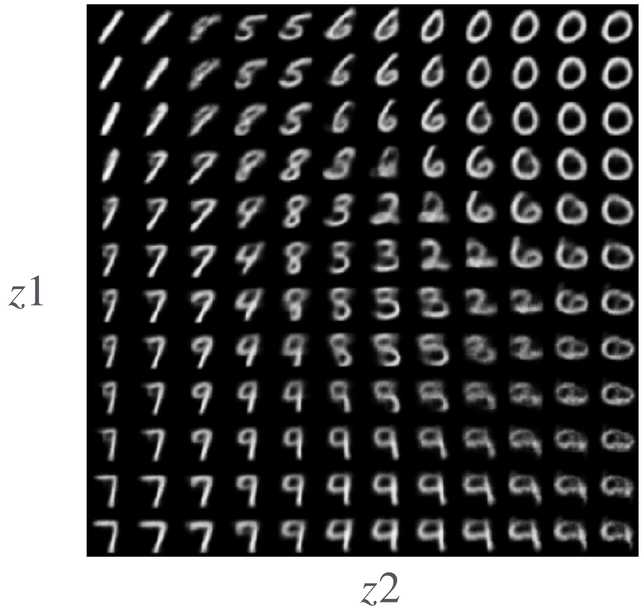
New samples:

We traverse the 2D latent space by scanning a grid of points:

$$z_{new} = (z_1, z_2)$$

Decoding each point as:

$$\hat{x_{new}} = g_\theta(z_{new}),$$



Reminder: Diffusion Model

$$x_{t^{(i)}}^{(i)} = \sqrt{\bar{\alpha}_{t^{(i)}}}\, x_0^{(i)} + \sqrt{1-\bar{\alpha}_{t^{(i)}}}\, \epsilon^{(i)}$$

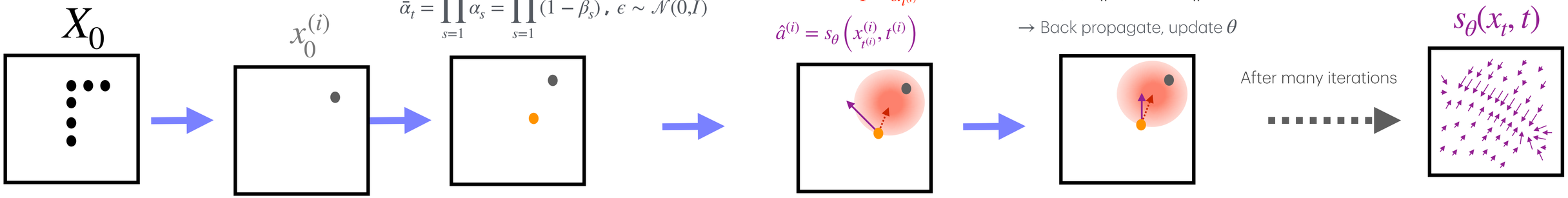
$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t} (1-\beta_s)\,,\ \epsilon \sim \mathcal{N}(0,I)$$

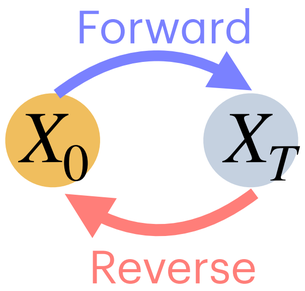
$$a^{(i)} = \frac{\sqrt{\bar{\alpha}_{t^{(i)}}}\, x_0^{(i)} - x_{t^{(i)}}^{(i)}}{1-\bar{\alpha}_{t^{(i)}}}$$

$$\hat{a}^{(i)} = s_\theta\left(x_{t^{(i)}}^{(i)}, t^{(i)}\right)$$

$$\ell^{(i)}(\theta) = \left\| \hat{a}^{(i)} - a^{(i)} \right\|^2$$

→ Back propagate, update θ





$$x_{t-\Delta t} = x_t + \left[\frac{1}{2}\beta(t)x_t + \beta(t)s_\theta(x_t, t)\right]\Delta t + \sqrt{\beta(t)\Delta t}\,\epsilon$$

x_0 $\quad$ $x_{t-2\Delta t}$ $\quad$ $x_T^{new} \sim \mathcal{N}(0,I)$

Conditional Diffusion: Similar to Diffusion, one extra input, c

$$x_{t^{(i)}}^{(i)} = \sqrt{\bar{\alpha}_{t^{(i)}}}\, x_0^{(i)} + \sqrt{1-\bar{\alpha}_{t^{(i)}}}\, \epsilon^{(i)}$$

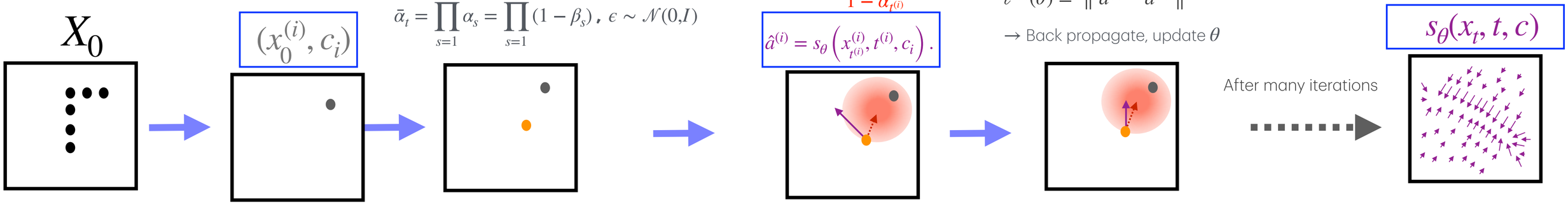
$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t} (1-\beta_s)\,,\ \epsilon \sim \mathcal{N}(0,I)$$

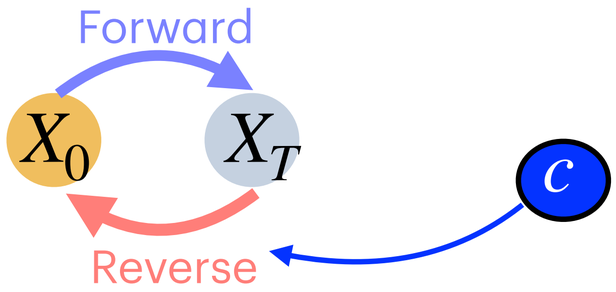
$$a^{(i)} = \frac{\sqrt{\bar{\alpha}_{t^{(i)}}}\, x_0^{(i)} - x_{t^{(i)}}^{(i)}}{1-\bar{\alpha}_{t^{(i)}}}$$

$$\hat{a}^{(i)} = s_\theta\left(x_{t^{(i)}}^{(i)}, t^{(i)}, c_i\right).$$

$$\ell^{(i)}(\theta) = \left\| \hat{a}^{(i)} - a^{(i)} \right\|^2$$

→ Back propagate, update θ





c_{desired} $\quad x_0$ $\quad x_{t-2\Delta t}$

$$x_{t-\Delta t} = x_t + \left[\frac{1}{2}\beta(t)x_t + \beta(t)s_\theta(x_t, t, c_{\text{desired}})\right]\Delta t + \sqrt{\beta(t)\Delta t}\,\epsilon$$

c_{desired} $\quad x_T^{new} \sim \mathcal{N}(0,I)$

Recipe Card: **Conditional Diffusion Models , Training**

Type of Learning: **Conditional generative learning**

Data set: **Paired data** $(x_0^{(i)}, c_i)$

Data

1- Load libraries: Basics + PyTorch

2-Load the paired Dataset: $\mathscr{D} = \{x_0^{(i)}, c_i\}_{i=1}^N$

-Each $x_0^{(i)}$ is one clean data example, c_i is its associated condition

3-Now set up the DataLoader for training/validation/testing:

-Each mini-batch must return: $\left(x_{0,\text{batch}}, c_{\text{batch}}\right)$.

forward process

4- Define the fixed forward process for one pair $(x_0^{(i)}, c_i)$.

4.1-Choose: $T =$ number of noise levels

4.2-Define: $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t} (1 - \beta_s)$

4.4-For one pair, $(x_0^{(i)}, c_i)$ where $i = 1, \ldots, N$

4.5-choose a random noise level: $t \sim \text{Uniform}\{1, \ldots, T\}$

4.6-Sample a Gaussian number from: $\epsilon \sim \mathcal{N}(0, I)$

4.7- Create one noisy version of $x_0^{(i)}$: $x_t^{(i)} = \sqrt{\bar{\alpha}_t}\, x_0^{(i)} + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$

-**Note that c does not enter the forward noising process.

Neural network

5- Build the score neural network

-The neural network receives: (x_t, t, c)

-It outputs a vector with the same shape as x_t : $s_\theta(x_t, t, c)$

-The goal is: $s_\theta(x_t, t, c) \approx \nabla_{x_t} \log p_t(x_t \mid c)$

-Architecture depends on the data structure.

Training

6- Calculate the score for mini batch $\{x_0^{(b)}, c^{(b)}\}_{b=1}^B$

-For one mini-batch $x_0^{(i)}$, $b = 1, \ldots, B$.

-For each $x_0^{(b)}$ the exact score is:

$$a^{(b)} = \nabla_{x_{t^{(b)}}^{(b)}} \log p_{t^{(b)}}\left(x_{t^{(b)}}^{(b)} \mid x_0^{(b)}\right) = \frac{\sqrt{\bar{\alpha}_{t^{(b)}}}\, x_0^{(b)} - x_{t^{(b)}}^{(b)}}{1 - \bar{\alpha}_{t^{(b)}}}$$

7- Define the score-matching loss:

-Predict the score via NN is: $\hat{a}^{(b)} = s_\theta\left(x_{t^{(b)}}^{(b)}, t^{(b)}, c^{(b)}\right)$

-For each $x_0^{(b)}$ the <u>stable</u> loss is: $\ell^{(b)}(\theta) = \left(1 - \bar{\alpha}_{t^{(b)}}\right) \left\| \hat{a}^{(b)} - a^{(b)} \right\|^2$

-Loss for a mini-batch: $\mathscr{L}_{\text{batch}}(\theta) = \frac{1}{B} \sum_{b=1}^{B} \ell^{(b)}(\theta)$

8- Training loop for each point in mini-batch $\{x_0^{(b)}, c^{(b)}\}_{b=1}^B$

8.1-Draw $\{x_0^{(b)}, c^{(b)}\}_{b=1}^B$, $\{t^{(b)}\}_{b=1}^B$ and $\{\epsilon^{(b)}\}_{b=1}^B$

8.2-compute $a^{(b)}$ and $\hat{a}^{(b)}$

8.3-compute $\mathscr{L}_{\text{batch}}(\theta) = \frac{1}{B} \sum_{b=1}^{B} \ell^{(b)}(\theta)$

8.4-Backpropagate $\nabla_\theta \mathscr{L}_{\text{batch}}(\theta)$

8.5- Repeat for many mini-batches and epochs.

9. Diagnostics after training

Recipe Card: **Conditional Diffusion Models, Generating New Samples**

Type of Learning: **generative modeling**

Data set: **Paired data** $(x_0^{(i)}, c_i)$

10- Load the trained diffusion model:

-Load score model: $s_\theta(x, t, c) \approx \nabla_x \log p_t(x \mid c)$

-Load the same forward noise schedule: $\beta(t)$

-Choose the number of reverse solver steps: K

-This sets: $\Delta t = \dfrac{T}{K}$

11- Choose a <u>desired condition</u> and start from noise:

- choose c_{desired}

-Pick a random initial point: $x_T^{new} \sim \mathcal{N}(0,I)$

12- Run the learned reverse SDE:

12.1-For $t = T, T - \Delta t, \ \ldots, \Delta t$. We start from $t = T$

12.2-sample the noise of SDE as $\epsilon_t \sim \mathcal{N}(0,I)$

12.3-Get the predicted score: $\hat{s}_t = s_\theta\left(x_t^{\text{new}}, t, c_{\text{desired}}\right)$

12.4-Then take one reverse Euler–Maruyama step: $$x_{t-\Delta t}^{\text{new}} = x_t^{\text{new}} + \left[\frac{1}{2}\beta(t)x_t^{\text{new}} + \beta(t)s_\theta\left(x_t^{\text{new}}, t, c_{\text{desired}}\right)\right]\Delta t + \sqrt{\beta(t)\Delta t}\,\epsilon_t$$

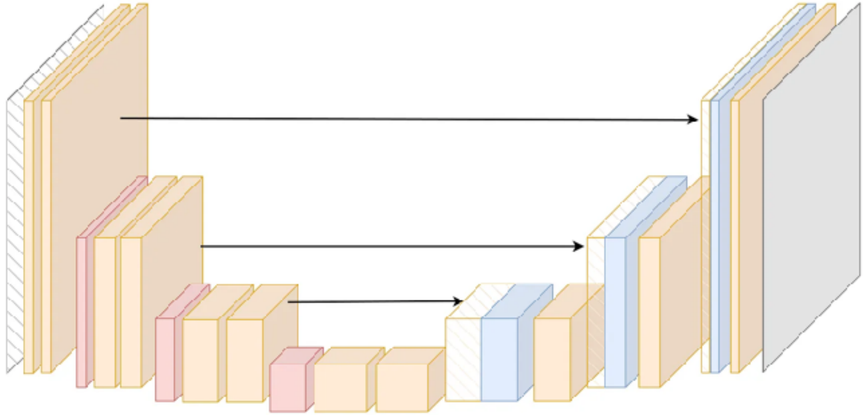
-Repeat (12.1-12.4) until getting to $t = 0$, and get $x_{t=0}^{\text{new}}$

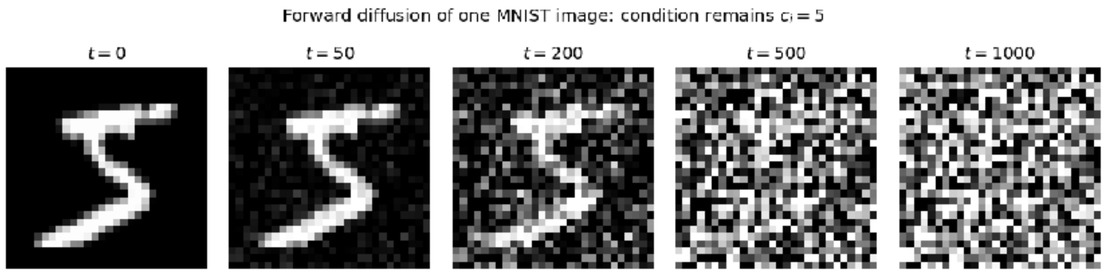
-Note: in last step we can use $\epsilon_{\Delta t=0} = 0$

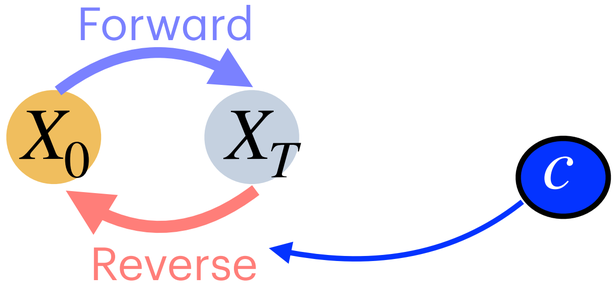
Example setup: MNIST dataset

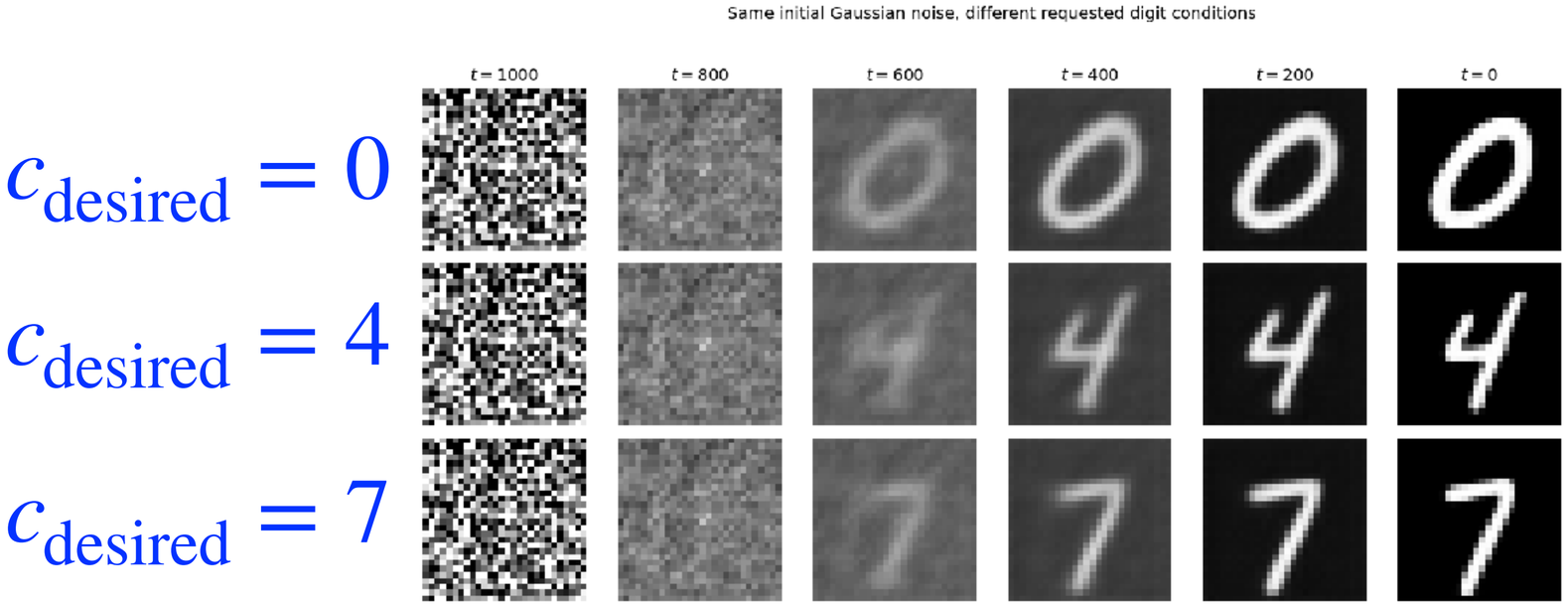
To estimate the score $s_\theta(x_t, t, c)$, we use

U-Net architecture

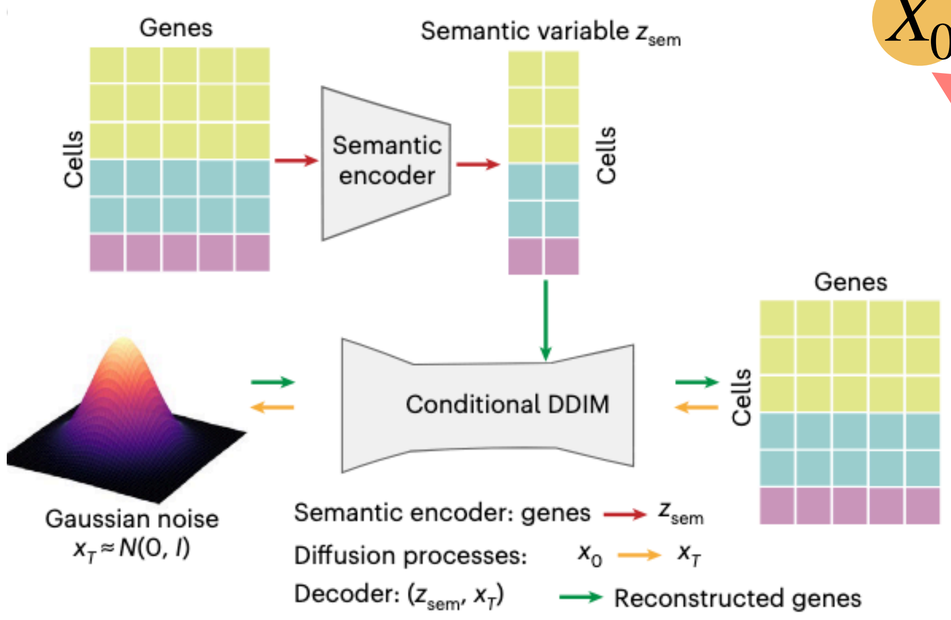


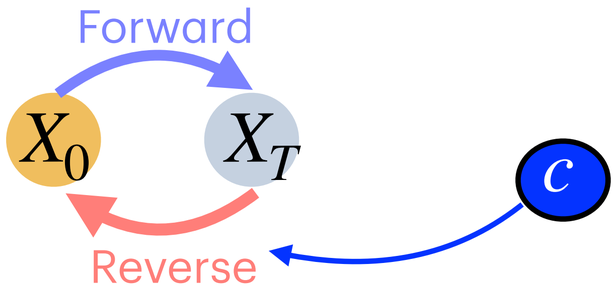


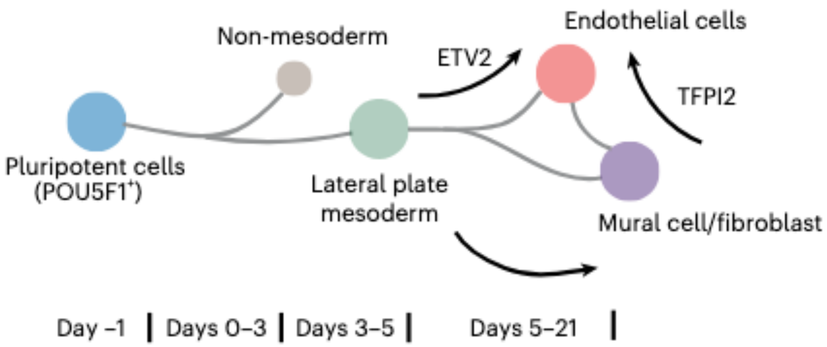


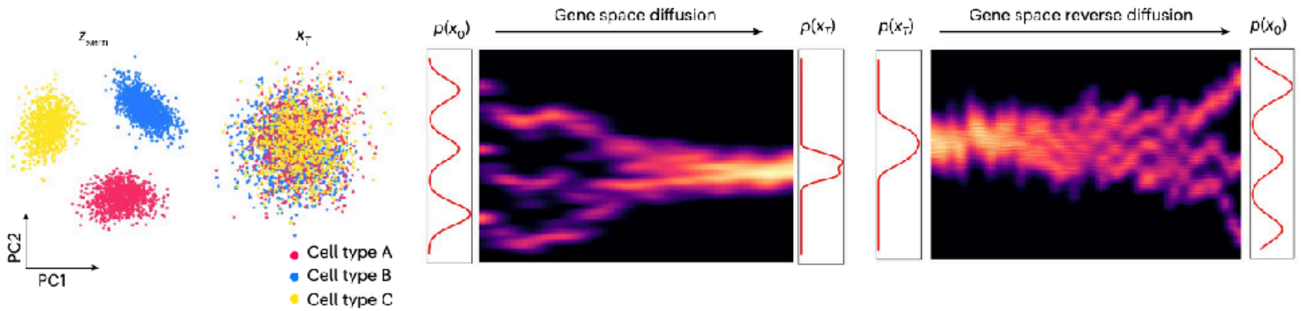


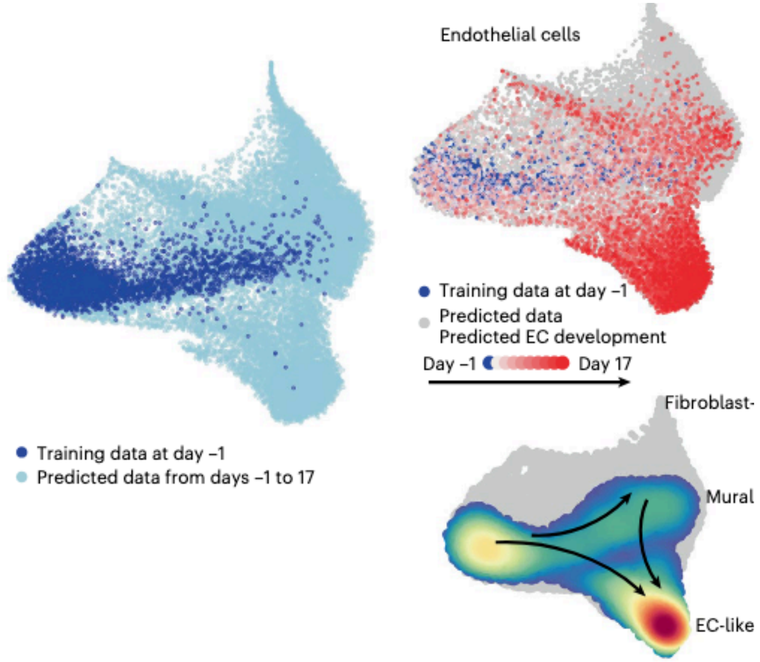
Example:





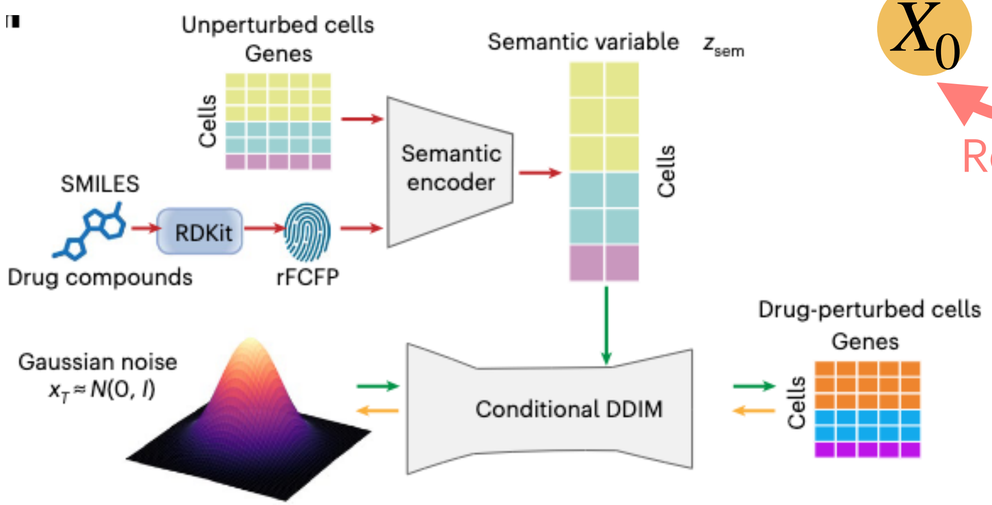


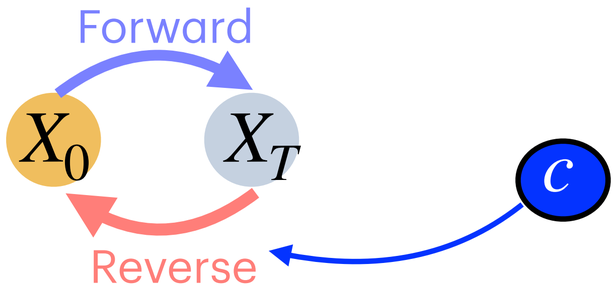




He, Siyu, et al. "Squidiff: predicting cellular development and responses to perturbations using a diffusion model." *Nature methods* (2026)

Example:





What Figure 4 actually does (from the text): They trained on just two timepoints' worth of **anchor data** — iPSCs at day −1 and the BVO cells at day 11 — and then *generated* the entire trajectory: "we combined scRNA-seq data from iPSCs and BVOs on day 11 and trained the Squidiff model on these datasets. We then performed interpolation for days 1, 3, 5, 7, 9, 13, 15 and 17." The figure caption is even sharper: the predicted day −1→17 trajectory "was learned by only endothelial cells on day −1 and day 11." So the model never saw days 1, 3, 5, … as training data — it *generated* them.

How that's possible — the exact mechanism (this is the key): It's the same latent-arithmetic trick as the perturbation figures, with *time* as the axis. From the Methods (the iPSC differentiation task, which is the template):

1. Encode the two anchor states into the semantic latent: z^0_{sem} (start) and $z^{\text{end}}_{\text{sem}}$ (end).
2. Form the **developmental direction vector** by subtracting them: $\Delta z = E(z^{\text{end}}_{\text{sem}}) - E(z^0_{\text{sem}})$. This single vector encodes "the average direction of differentiation over the whole period."
3. **Generate any intermediate day by stepping a fraction along that vector:** $z^i_{\text{sem}} = z^0_{\text{sem}} + \frac{i}{N}\Delta z + \eta$, then decode with the conditional DDIM. Walking i from start to end sweeps out days 1, 3, 5, … 17.

He, Siyu, et al. "Squidiff: predicting cellular development and responses to perturbations using a diffusion model." *Nature methods* (2026)

for awareness: For awareness: Direct conditional diffusion does not always **steer samples toward** c as strongly as we want. Two approaches can strengthen that steering:

Guidance: pushing harder toward c by amplifying the conditioning at generation by adding a scaled "steering arrow":

$$s_{\text{guided}}(x_t, t, c) = s_{\text{base}}(x_t, t) + w\, g(x_t, t, c)$$

1. Classifier guidance: the steering arrow comes from a separate, differentiable classifier or property predictor. It can steer a frozen unconditional diffusion model toward a desired property.

$$g(x_t, t, c) = \nabla_{x_t} \log p_\psi(c \mid x_t, t)$$

2. Classifier-free guidance: one network learns both conditional and unconditional scores. The difference between them is the steering arrow; no separate classifier is needed.

$$g(x_t, t, c) = s_\theta(x_t, t, c) - s_\theta(x_t, t, \varnothing)$$

Motivation for Geometric Deep Learning: generate new potential drug molecules

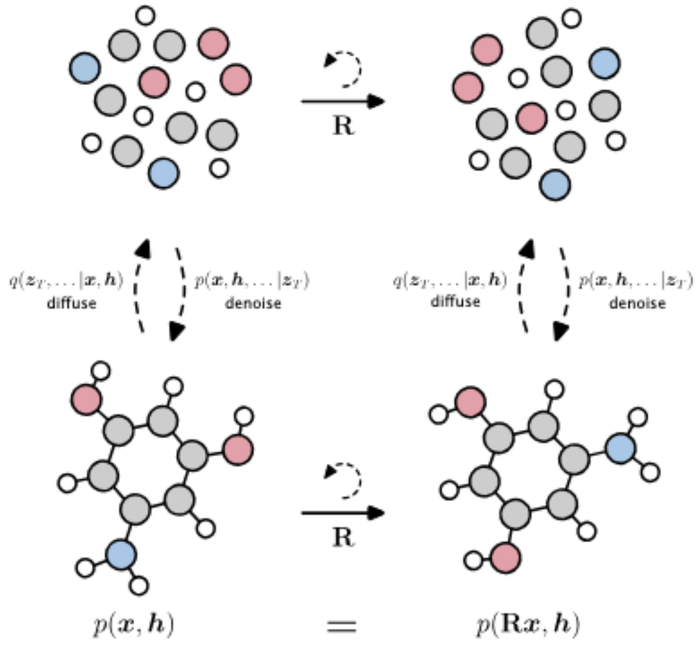
Diffusion + Geometric Deep Learning

Equivariant Diffusion for Molecule Generation in 3D

Emiel Hoogeboom [*1] Victor Garcia Satorras [*1] Clément Vignac [*2] Max Welling [1]

Abstract

This work introduces a diffusion model for molecule generation in 3D that is equivariant to Euclidean transformations. Our E(3) Equivariant Diffusion Model (EDM) learns to denoise a diffusion process with an equivariant network that jointly operates on both continuous (atom coordinates) and categorical features (atom types). In addition, we provide a probabilistic analysis which admits likelihood computation of molecules using our model. Experimentally, the proposed method significantly outperforms previous 3D molecular generative methods regarding the quality of generated samples and efficiency at training time.



1. Introduction